AI Architecture Design

Session 1 - Architecture Views, Steps and Process. A complete study guide: every slide, plus the full session walkthrough, war stories and Q&A.

Instructor Eng. Sameh Amin - Unifonic | MLOps MENA
Session 1 of 6 - 18 October 2026 - 1h 54m
Project Marketing Automation with AI & Agents
Compiled from the 36-slide deck and the session recording

Contents

How to use this guide 4

- Conventions used here 4
- The one-sentence summary of Session 1 4
- What you should be able to do after studying this 4

Part 0 — Course map, logistics and operations 5

- 0.1 What the course is — Slide 3 5
- 0.2 What the course is not — Slide 4 5
- 0.3 Why this matters for your career — the opening remarks [00:02] 5
- 0.4 Course setup and logistics — Slide 5 5
- 0.5 Course operations 6
- 0.6 The six-session roadmap — Slide 2, expanded at [05:25] 6

Part 1 — The architect: role, personality and culture 7

- 1.1 Is "architect" still a job title? [14:43] 7
- 1.2 The eight traits — Slide 6, with the instructor's commentary 7
- 1.3 Make your hands dirty [45:36] 8

Part 2 — The project: Marketing Automation with AI & Agents 9

- 2.1 Functional use cases — Slide 8 9
- 2.2 KPIs and objectives — Slide 8 9
- 2.3 Non-functional use cases — Slide 8 10

Part 3 — The business requirements 12

- 3.1 Stakeholders / personas [28:40] 12
- 3.2 Use cases assessment — the planning call [29:26] 12
- 3.3 Should you give timelines now? Ready for implementation vs spikes [30:58] 12
- 3.4 The Why/What vs How/When split [32:30] 13
- 3.5 Should we stop the architecture description until all spikes complete? [31:43] 13
- 3.6 Closing the gaps, questions and inquiries (1–n rounds) [43:18] 14

Part 4 — Architecture frameworks 15

- 4.1 TOGAF — Slide 11 15
- 4.2 Zachman — Slides 12-13 16
- 4.3 The C4 model — Slide 14 16

Part 5 — The architecture development process 18

- 5.1 The five steps 18
- 5.2 The outputs — what you are actually accountable for 18
- 5.3 The loop [43:18] 18

Part 6 — Views and perspectives 19

- 6.1 The six views — Slide 16 19
- 6.2 The perspectives / quality attributes — Slide 16 19
- 6.3 The cross-section technique — the key idea of the session [50:20] 19
- 6.4 The budget consequence — why this is the architect's job [52:40] 20

Part 7 — Architecture and Agile 22

- 7.1 From waterfall to sprints [55:01] 22
- 7.2 Does agile change the architecture? [55:50] 22
- 7.3 The four levels on slide 18 22
- 7.4 Principles as the stable layer [56:35] 22

Part 8 — The architecture description 23

8.1 Architecture intro and facts — Slide 20 23
8.2 High-level design — Slide 21 23
8.3 Information viewpoint / data pipelines — Slide 22 23
8.4 Concurrency viewpoint — Slide 23 24
8.5 Operational viewpoint — Slide 24 24

Part 9 — The live design walkthrough 25

9.1 Document skeleton 25
9.2 Problem statement — "What we are trying to resolve" [58:12] 25
9.3 Goals and non-goals [59:45] 25
9.4 As-is architecture [62:04] 26
9.5 Architecture principles and key decisions [70:36] 26
9.6 To-be architecture — the layers that get added [81:39] 28
9.7 Use case 1 — Segmentation (the base for everything else) [84:02] 28
9.8 Use case 2 — Personalisation / recommendation engine [88:39] 29
9.9 Use case 3 — Abandoned cart / next-best action [90:13] 29
9.10 Use case 4 — Churn prediction [94:52] 30
9.11 Use case 5 — Cold start [99:31] 30
9.12 Use case 6 — Agentic marketing 30
9.13 What the course output is [102:36] 31

Part 10 — Architecture governance 32

10.1 The five key points — Slide 26 32
10.2 Where the documents actually live [107:59] 32

Part 11 — Architecture & agentic coding 33

11.1 Do it manually first 33
11.2 Never fully autonomous — always plan mode 33
11.3 Skills to give the agent 33
11.4 The division of labour with C4 33
11.5 Where agentic-coding cost belongs [103:22] 33

Part 12 — Q&A from the session 35

Part 13 — W1 mini project (assignment) 37

13.1 The brief 37
13.2 How to attack it, using this session's material 37

Part 14 — Glossary of the session's vocabulary 38

Part 15 — Self-check before the quiz 39

Appendix — Session 1 at a glance 40

How to use this guide

ORIENTATION

This document is a full write-up of **Session 1** of the AI Architecture Design course: everything on the 36 slides, plus everything the instructor said in the 1h 54m recording that is *not* on the slides — the war stories, the rules of thumb, the live design walkthrough he did on draw.io, and the Q&A at the end.

The session was delivered in Arabic with English technical terms. This guide is written in English, and technical terms are kept exactly as the instructor used them, so the vocabulary carries over to your interviews and your design docs.

Conventions used here

Marker	Meaning
Slide N	The point appears on slide N of the deck.
`[mm:ss]`	Timestamp in the recording, so you can jump to the spot.
Teal callout	A rule, definition or principle worth memorising.
Orange callout	A story, a trap, or a mistake the instructor saw in production.

The one-sentence summary of Session 1

> Architecture is not a diagram — it is a **process** that starts at the business pain point, runs through a set of **views** (functional, information, concurrency, deployment, operational), is cross-cut by a set of **perspectives** (security, performance, availability, evolution, regulation), and ends in a written **architecture description** plus a skeleton you built with your own hands.

What you should be able to do after studying this

1. Explain the difference between a **view** and a **perspective**, and apply a perspective across views to surface gaps nobody asked about.
2. Run a business-requirements session: identify stakeholders and personas, capture use cases, separate functional from non-functional requirements, and close the gaps over 1–n rounds.
3. Decide when to give timelines and when to ask for a **spike** instead.
4. Draw a C1 (context) and C2 (container/deployment) view for an AI system.
5. Write the standard architecture-description headings: problem statement, goals/non-goals, as-is, principles and key decisions, assessment, priorities.
6. Translate a model metric (accuracy, F1) into a business KPI (CSAT, take-rate) that a non-technical stakeholder will sign off on.
7. Produce capacity planning and a budget, not just boxes and arrows.

Part 0 — Course map, logistics and operations

SLIDES 1-5, PLUS THE OPENING

0.1 What the course is — Slide 3

- A **general architecture method, applied to AI systems**. The method is not AI-specific; the application is.
- AI design best practices.
- **Practical sessions with enterprise case studies** — telecom, e-commerce, government — for each session.
- Attention to **functional and non-functional quality attributes**, so the result is production-ready.
- Target audience: senior engineers from any specialisation (BE, FE, AI, …), EMs, TLs, up to CTOs, plus semi-technical people and PMs.

> The instructor's framing `[04:39]`: "We can't jump straight into AI. We have to start with an introduction to architecture design, or design in general." The material comes from two sources mixed together — books on system design, clean code and clean architecture — **plus** applying them in reality. "You don't always find that what's in the books can be applied in reality. And what works in Europe doesn't necessarily work in the Gulf, and what works in the Gulf doesn't necessarily work in Egypt. The country itself makes a difference."

0.2 What the course is not — Slide 4

- Not a product/business-ideas course, and not for business-only people.
- Not a machine-learning or data-science course. `[11:00]` "There are people better than me in that track who can teach it; it's not the goal here."

0.3 Why this matters for your career — the opening remarks `[00:02]`

The host's framing, which is worth keeping in mind for the whole course:

- Coding itself has become easier with agentic tools (Claude and others). **What separates a senior from everyone else now is architecture design.**
- Almost everyone in the room had faced an **AI system design interview** and found it the hardest stage.
- The instructor confirmed the loop closes: the system design done in an interview was later **implemented live at the company**, component for component `[03:53]`.

> There is no single "right" answer in system architecture. `[04:39]` "System architecture is not a fixed thing — there are many schools and many views. Every company has its own principles, its own boundaries, its own rules. Something being right at company A does not make it right at company B."

0.4 Course setup and logistics — Slide 5

- **UML / diagram tool:** Draw.io, Mermaid, or paper.
- **Any OS** (the instructor is on a Mac).
- **Wiki tool:** Confluence, Obsidian, MS Word, Google Docs — whatever you write your design doc in.
- **Agentic coding IDE** (optional).
- **Excel** (MS or LibreOffice) — needed later for capacity planning and sizing.

The three-tab working setup `[11:38]`

The instructor keeps **three things open at all times** while designing, and recommends you do the same:

1. **The business requirements** — the Confluence page, Jira epic, or whatever holds the use case you were given.
2. **The diagram tool** (draw.io) — where the views get drawn.
3. **The design document** — where every decision, principle and open question lands.

> Why three tabs and not a linear write-up `[12:24]`: "The design document is **not** filled in from top to bottom. It would be lovely if it were. But while you're designing and discussing with the team, something comes up — and you don't want to lose it. So you go to the security section and write it there. An architecture principle comes up mid-discussion — you put it there. From experience and from many meetings: we say excellent things in meetings, then everyone goes their own way, nobody wrote anything down, and it's gone. Always document the thing, even if you won't act on it now."

0.5 Course operations

Item	Detail
Sessions	6 total
Mid-course	A break week with a recap and a mid-course quiz
Quiz	After every session, MCQ, one week to answer, on the academy platform
Mini projects	Optional, corrected, with feedback returned to you
Final project	Required for the certificate
Attendance	No more than **2 absences**
Channels	Telegram study group (private, for enrolled students) + a public study group; YouTube comments answered in the next session
Recordings	Sessions are published on YouTube
Second cohort	Planned for people who registered but couldn't attend the first cohort

0.6 The six-session roadmap — Slide 2, expanded at `[05:25]`

#	Session	What gets added to the design
1	Architecture Views, Steps and Process	The base: method, views, perspectives, the project's as-is architecture and principles
2	AI System Building Blocks	Data pipelines, data architecture, training vs inference, fine-tuning, feature stores
3	LLM & RAG Architecture	Adding LLM usage (text generation and other tasks) plus RAG architecture to the design
4	Agentic AI Architecture	Turning understanding into workflows that take action
5	Production Readiness	Security, governance, evaluation, monitoring
6	Enterprise Case Studies	A lead-in to the capstone project

> The course is **project-based** `[05:25]`. One use case runs through all six sessions, and each session layers its topic onto the same growing architecture document. You are building one architecture together, incrementally — not six disconnected lectures.

Part 1 — The architect: role, personality and culture

SLIDE 6

1.1 Is "architect" still a job title? [14:43]

- It used to be a distinct profession. The instructor's own title at Etisalat was **Senior Solution Architect**.
- Now that titles have merged, the architecture work is usually done by the **Team Lead** or the **Engineering Manager**.
- At larger scale, the **Director of Engineering** owns the **enterprise architecture** that everyone else follows.

> Being a people manager does not mean you can architect. [15:30] "It doesn't follow that because I'm a people manager, I know how to do architecture with people, or that I'll produce a successful architecture." It takes a set of principles — and some changes to your personality.

1.2 The eight traits — Slide 6, with the instructor's commentary

1. Humble — puts the best solution ahead of his own idea

[15:30] "You must accept that someone with two years of experience, while you have fifteen, can say a good idea that works and comes out well. Accept it. Don't be arrogant. You'll validate it, of course — you'll check whether it's right or wrong, maybe let them try it and come back to you — but in the end you have to give them their chance."

2. Curious — keeps learning new technologies and domains

[16:16] "If you're in the technical field, **half an hour of reading a day** on new areas, following the news, new technologies — on the weekend or any time. This isn't optional; it's our reality. Install things, try agentic coding, try local or open-source LLMs."

3. Pragmatic — picks the simplest design that meets the requirements (MVP)

[17:04] "We don't have to build the whole system at once — we can do an MVP first. But when you think about the MVP, **think about it in 360 degrees**: not functional only, but secured, performant, and compliant with the rules of the country you're building in."

> Going out with a **small but complete and correct slice** is better than going out with a large functional slice that collapses. It also protects you later: minimise the scope, get the non-functional side right, rather than shipping wide and discovering defects after the feature has been rolled out to users — at which point changing it is very hard. [10:06]

4. Technology agnostic — isolated from his preferred tech

[17:50] "Drop the tool from your head while you think about the architecture. Keep it **abstract**. Then, if your preferred tool doesn't fit, you can propose a different one to your manager or to the company's architecture committee."

> The anti-pattern, stated plainly [18:36]: "I will not use Postgres just because we have it and I'm used to it. That is a **wrong decision**." The correct order is: know what the right thing is, *then* size it against the tools the company already has — not the reverse. If you proposed Oracle and the company said no, fine, now you adapt to Postgres consciously.

5. Trade-off thinker — no design is perfect, every choice has a cost

[18:36] "In solution sessions people notice we don't talk about what's *correct*. We say: what are the **options**? Option one, two, three. What are the **pros and cons** of each? Then we study them against our company, our customers, our country — and in the end we pick one or two, and that is what we design against."

6. Strong communicator — explains the same design to engineers and executives

[19:22] "If I'm going to sit with a CTO to explain an architecture, I don't take the same document I'd take to the team. The infra/IO people have their own view. The business has another."

- **CTO**: wants one picture with everything on it, and five minutes on what the system does, its limitations, and what support you need from them.
- **DevOps / infra**: wants the network, the connections, the ports — the detail.
- This is the formal idea of **architectural viewpoints**: each audience has the view it cares about.

7. Collaborative — listens first, influences without authority

[16:16] But do not take the other extreme either: "Don't be a yes-man. Sometimes you need a **hard no** — and the hard no must come with a *why*. You have to convince them to some degree: 'no, this won't work because of 1, 2, 3' or 'we tried it before and it didn't work'. Otherwise the team is demotivated because you never give them a chance. It has to be a balance."

8. Accountable — owns decisions and their consequences (POC, skeleton)

[20:09] "In the end it's your solution. If there are gaps, it's your responsibility — though it's the whole team's responsibility, not yours alone. **Don't let that scare you out of taking decisions.** We all make mistakes, we all end up with solution gaps, and we learn over time."

> The habit to build from this: keep a **lessons-learned checklist**. Every mistake you fell into goes on the list. Before any new design ships, you run it against the checklist. Over time it accumulates. You can also hand that checklist to your AI assistant so it learns from it too.

1.3 Make your hands dirty [45:36]

A cultural point the instructor returns to twice:

> "There's a concept in our region especially: 'I've reached a certain level, I don't want to write code, I don't want to open an IDE, I'll write a paper and that's it.' No. I currently see directors and CTOs writing programs."

Two reasons to build a **skeleton / POC / MVP yourself** after the design is agreed:

1. **You discover your own design errors.** "You yourself may discover things that are wrong in your design, go back, and fix them." [46:25]
2. **Speed.** If you hand it to the team for a full cycle, feedback comes back in two or three weeks or a month. If you do a **dry run** with your own hands, you find it and fix it immediately.
3. And a third, social one: **the team must feel your hands are in it.** "I'm not an architect who shows up, hands you a paper, and leaves." That is how you earn the team's respect.

Part 2 — The project: Marketing Automation with AI & Agents

SLIDES 7-8

This is the single use case the whole course is built on. It is **assembled from several real companies**, not one `[20:55]` — telecom, e-commerce and government work the instructor did between roughly 2012 and today (computer vision, recommendation engines, chatbots, SMS and email classification, then RAG).

> Scope note `[20:55]`: it is broader than a recommendation engine. "Marketing" here includes agentic AI, a recommendation/personalisation engine, and data pipelines underneath.

2.1 Functional use cases — Slide 8

A. Homepage / products page personalisation (feature stores)

`[21:42]` When you open any app — Amazon, a telecom app — you see deals and offers on the home page. Open it on someone else's account and it is completely different. That difference is personalisation, computed from **your interactions, your history, your profile**.

> **The production story that defines this use case** `[21:42]`: men started receiving women's offers. The investigation found that people register telecom lines for their wives (buying them phones with their own ID), or the reverse — so the person is registered in the system under the other gender. The system was "understanding" the profile as female.
>
> The fix was to stop trusting the registration field and **infer from behaviour instead**: "whoever buys a laptop bag like that is never going to want a women's perfume." Behaviour beats declared profile data.

B. Next-best action

The canonical example is the **abandoned cart** `[22:28]`: a user fills a cart, decides it is expensive, and leaves it. You detect that and send them a nudge. More generally, next-best action means: *see where the customer is standing right now and decide what to do about it*. Another case: a user who hasn't bought or logged in for a period — detect it and send an offer that brings them back.

C. New users (cold start)

`[23:13]` A brand-new user: you know nothing about them, and they know nothing about you. Approaches discussed:

- A **fixed list** of preferred offers driven by rules.
- **Segmentation** — find who they resemble on the attributes you do have, and treat them like that segment.
- **Ranking** on top of either.

D. Agentic marketing — campaign automation

`[23:59]` A case the team was working on at the time of recording: using agentic AI to **orchestrate the marketing campaign** end to end, LLM plus agentic AI. This is deferred to Session 4.

> These four use cases are enough to carry the entire course. Later sessions bolt their topic onto them one at a time.

2.2 KPIs and objectives — Slide 8

Slide targets: increase the sales take-rate by 80%; increase CSAT to 70%; a feedback loop.

The lesson behind the KPI slide [24:45]

> The instructor was sitting with a Saudi customer and was asked: *what will tell us this system succeeded or failed?* He answered with model accuracy — "it reached 90%." The customer's reply: **"90% of what? What does 90% mean? How do I even measure that?"**
>
> "The mistake was mine. He isn't a technical man. I can't tell him accuracy or F1 score — that's not his. But they do understand **CSAT**."

How business actually measures it [25:30]:

- The **feedback** that appears after each transaction — the star rating, 4 out of 5, 5 out of 5.
- Other **indirect indicators** they already collect to tell whether people are happy.
- They want to know: *did the new chatbot raise people's satisfaction or not?*

> **The rule:** in the design document, separate the technical metrics from the business KPIs, and show **how your pipeline converts one into the other**. "How do I take the metrics I have and deliver them to the business as CSAT, or as sales?"

Writing a KPI properly [25:30]

For each KPI, record three things:

1. The **current value** ("sales are at 60% today").
2. The **target** ("I want 80%").
3. The **contribution of your system** — how much of the delta this system is supposed to produce.

> Why this is not bureaucracy [26:17]: "Everything you work on must be linked to the business, so that at least at the end of the year what you did counts as effort you made — not something hanging in the air."

The feedback loop [26:17]

Most people ship the system and stop. You need a mechanism that collects signal back:

- Thumbs up / thumbs down, like ChatGPT.
- Free-text feedback and written returns/complaints.
- Then automate collecting it, or study it, and **use it to improve the design**.

In this project, the feedback loop is concrete: take the star rating, run **classification over the free-text feedback** to label it satisfied / not satisfied, combine it with the rating, and you have a simple "is this person happy or not" signal [61:17].

2.3 Non-functional use cases — Slide 8

Non-functional requirement	What to pin down
Latency	Real-time, near-real-time, or batch — per use case, not for the whole system
Scalability	Number of users, peak campaigns, seasonality
Explainability and business control	If the user hits an error, or business asks "why did the AI produce this number?", you need an answer
Security & compliance	Data residency, privacy (PDPL), consent

Why non-functional requirements are the dangerous ones [09:18]

> "The functional attributes — the business will usually tell you. The non-functional ones — usually **nobody will tell you**. So one of two things happens: either you discover or anticipate them before production, or they come to you as an incident **in** production."

And the regional observation `[09:18]`: "In our part of the world we tend to want to run fast and ship fast, so we skip a lot of non-functional requirements — security, performance — with 'let's ship and improve it later'."

Explainability, concretely `[27:07]`

"If the user hits an error, how is your system able to explain? It's as if you're debugging, but from the business side. He asks you 'how did the AI produce this number?' — you must have an answer ready: it came from this calculation, this rule."

Security & compliance — the data residency rule `[27:53]`

> "In Saudi or in the UAE, the data cannot leave. Meaning: I can't use a model hosted in America. Not **on transit** and not **on disk**. Some people say 'I won't store the data' — that's not enough. Even in transit, even on the network, the data must not leave.
>
> **There are companies paying hundreds of thousands of dollars because they violated these rules.**"

Two practical consequences:

1. If there is a security team in your company, your application must be **certified** by them before it ships.
2. Be **proactive**: ask for these requirements early. "It will change decisions in your design. It makes a big difference if you know it from the start — you won't be able to do it afterwards."

Part 3 — The business requirements

SLIDE 9

The five items on slide 9, each expanded from the session.

3.1 Stakeholders / personas [28:40]

You need to know **who will use the system**:

- Is the consumer a human, or **another system**? "Sometimes they tell you: my system here will be called by another system. That's a different case."
- Is that system **inside or outside** the company? That is a decision with security consequences.
- What **persona** is each human user — a marketer, an accountant, a CEO? "This matters a lot."
- For every user type, know which **features they are allowed and not allowed** to use.

3.2 Use cases assessment — the planning call [29:26]

Let them dream — then write it down

> "I have no problem with business dreaming. Let them say all their dreams. In that dream phase, let them — buy everything they want to say."

But: **verbal is not enough.** "It's lovely to take it verbally, but more important is that it is **written**." When you come to manage it, you need epics, stories, or a Confluence document, so that you agree on:

- what the **expected input** of the system is;
- what the **acceptance criteria** are — how *he* will judge whether this application failed or succeeded.

> The mental model [30:13]: "Consider the product manager or the business person **your customer**. It's exactly as if you were building a system for Carrefour — he *is* Carrefour. I need to satisfy this man."

What you must agree on explicitly: the KPIs, who will use the system, the personas, and the feature permissions per user.

3.3 Should you give timelines now? Ready for implementation vs spikes [30:58]

> "He gave you the dreams — giving timelines straight away is wrong, in my view. Unless you're technically certain because you've done it a million times before."

The correct sequence:

1. Take your time, **study the subject**.
2. Surface the **gaps** and the **questions**.
3. Sit once, twice, three times until you close them.
4. **This is the first stage of doing any architecture: closing the requirements.**

> Have the **whole team** attend these sessions — you may have no questions, but the team will have other questions.

Splitting the work into "clear" and "needs a spike" [31:43]

After an initial high-level architecture, you negotiate with the PM:

- **"These parts are clear to me"** — they can ship quickly.
- **"These parts need research"** — e.g. *"we've never done agentic AI before, we need to investigate; this will take two weeks to a month to explore and come back."*

> "In the end **you don't take business decisions on behalf of the business person**. If it's his decision, go back to him and let him take it. The product manager is the CEO of the project. I'm a technical man — I can tell you *how* it gets done."

3.4 The Why/What vs How/When split [32:30]

Owner	Owns
Business / PM	**What** we are going to do, and **Why** we are doing it
Engineering / architect	**How** we will do it, and **When** it will be done

> "Don't mix the two, because that's where complexity and wrong deliveries come from — we deliver something other than what we agreed on. It happens when **the two roles enter each other**: I take business decisions implicitly on his behalf without telling him, or he interferes in my technical decisions and tells me how to proceed. **Both are wrong.**"

This does not forbid suggestions in either direction [33:17]:

- You may propose a product change — "what do you think, instead of that screen, we do it this way?" — but **he is the owner**. If he says "I want it this way", that's his call. **Document it.**
- He may propose a technical decision — you take it seriously, and you may still say "I see that we use what the company already has", because **that decision is yours**.

3.5 Should we stop the architecture description until all spikes complete? [31:43]

No. Nobody will wait a month while you go silent. The work is interleaved: initial high-level architecture, then negotiate which pieces are clear and which need a spike, keep designing the clear parts.

Feasibility study — and what to do with impossible dreams [44:04]

> **The fire alarm story.** A large company asked for AI that detects whether a fire alarm is true or false. "If the fire brigade can't tell, how would the AI? How will it know whether this is shisha smoke or gas-stove smoke?" The project failed — and failed fast. A colleague went and watched videos of how people in hotels trick alarms, to figure out how the model could even be trained. "So you must have a **feasibility study**: some of these dreams work and some don't."

The practical split [47:10]:

1. **Obvious cases** — "clear as the sun, you've done it before, you know it won't work." Highlight it immediately. No need to go do an architecture and come back.
2. **Cases needing research** — these go to a **skeleton or POC**, because you need some research.
   - If the product team grants you time to research: go explore and come back.
   - If they don't: say so honestly — *"I don't know it, so we'll try it together."*
   - A middle path: take two days, look at what people outside are doing, come back.

3. **Either way, highlight the risk** [47:55]: "Don't put assumptions in. Tell the business people: this is the first time the team does this, we're exploring together."

Feature flags and rollout — the risk-sharing mechanism [47:55]

Once a risk is highlighted, the PM has tools:

- Try it on the side and ship it behind a **feature flag** — "the feature flag thing is the best."
- Ship the parts you're confident about first.
- For the uncertain parts, do a **limited rollout** — "like Google does, like Microsoft does: release to a limited audience, five or ten companies, a small scale of testers."
- If it works → **GA (general availability)** to everyone. If it fails → it comes back to you and you work on it until it ships.

3.6 Closing the gaps, questions and inquiries (1–n rounds) [43:18]

The loop: take the principles, the assumptions, the outputs and questions → sit with each stakeholder → align → collect feedback → feed it back into your design → repeat once, twice → the solution is closed and everyone is satisfied → **now** you sign off on implementation.

> "Measure twice and cut once." [34:04] "In most cases you'll never close a design or a solution from the first time. But you try."

And the cost of skipping it [44:04]: "The big problem is 'come on, quickly, let's finish' — and in the end we're surprised that we built something completely different, and we ignored things that will hit us later. And in the end, unfortunately, **you are blamed**: you're the one who did the design, how did this pass you? You don't do your work so that nobody blames you — you do it because it's the right thing."

Part 4 — Architecture frameworks

SLIDES 11-14

Three frameworks are introduced. The instructor's position: learn the basics of all of them, then **pick and mix** [34:50].

> "When you have the basics right, you start to pick and match the things that serve you and work with them. Don't say 'these are theoretical'. Even now, in AI, we use the simplest machine-learning rules in very advanced things. **Don't underestimate the simple concepts or the basics** — in the end you will need them."

4.1 TOGAF — Slide 11

- **The Open Group Architecture Framework.** A methodology: *the process to create and deliver architectures.*
- Four layers: **Business, Data, Application, Technology**.
  - Business Architecture — business processes, organisation, people.
  - Application Architecture — services.
  - Data Architecture — data, information.
  - Technology Architecture — hardware, software, network.
- **ADM** — Architecture Development Method.
- It was produced by a consortium of large companies (Microsoft, Oracle and others) agreeing on a common framework [35:37].

The one idea to take from TOGAF: start from business [35:37]

> **"Don't start from technology — start from business."** The instructor heard this early in his career at a Google conference: *"don't start technology from business"* — look for the **pain point**, and that's what you start working on.
>
> "There is no such thing as 'agentic AI' to sell. I'm not going to go to Carrefour and say 'agentic AI'. He'd say 'what do I do with it?'"
>
> "That's why you find tiny, cheap, simple programs — built by people who aren't even deeply technical — that are very successful. He found a pain point and he's solving it."

The as-is walkthrough — how you actually apply the business layer [37:10]

> **The charity / medical-treatment example.** Going into agentic AI around 2024, sitting with Saudi and Egyptian companies, the instructor's first move was never to sell the technology — it was to **understand the current process**:
>
> "Understand what you have *now*: the process, the workflow, the system as it is, **with its errors**, with everything. Then: what is the pain point you have? Why do you want a system at all?"
>
> At a charity, people apply for medical treatment. Many manual stages: a person takes the prescription, scans it, confirms it is correct. That's an OCR system — "except no OCR can read doctors' handwriting, which is the black hole we all fell into." So **that stage must stay human**.
>
> "You walk with him and he tells you his pain points. You find very long stages. Then it starts to light up in your head: *this one could be OCR, this one could be a learning system to classify the specialisation*."

This is the **as-is architecture** step, and it is the reason the design document has an "As-is" heading. [38:41] "You're coming from solid ground. You're actually standing on the problems and you know what exists. I'm not going to take two words and go say 'here's a system this big' and in the end he does nothing with it."

Then you descend the TOGAF layers: application architecture → data architecture → and at the very end, the tools (the hardware/technology layer).

4.2 Zachman — Slides 12-13

- Focuses on **what to create and deliver** (TOGAF focuses on the *process*).
- A 6 × 6 matrix: six fundamental questions — **What (Data), How (Function), Where (Network), Who (People), When (Time), Why (Motivation)** — mapped to six perspectives: **Planner, Owner, Designer, Builder, Implementer (Sub-contractor), User**.
- Each cell is a concrete artefact: e.g. *Data* × *Owner* = semantic model; *Function* × *Designer* = application architecture; *Network* × *Builder* = technology architecture; *People* × *Sub-contractor* = security architecture.
- The rows read as levels of abstraction: Scope (contextual) → Enterprise Model (conceptual) → System Model (logical) → Technology Model (physical) → Detailed Representations (out of context).

The instructor's honest assessment `[39:26]`

> "Nobody uses it much now, because to do all of it takes a very long time. It's mostly done for **enterprise** things — if someone is designing the enterprise architecture for a big telecom company, not for our day-to-day operational systems."

Still, the value is the **dimension-by-dimension sweep**: for each cell you ask what/how/where from the angle of people, network, function, time — and you start to see which view is appropriate for which audience.

4.3 The C4 model — Slide 14

By **Simon Brown** — "maps of your code".

Level	Contains	Audience
C1 — System Context	People, the system, external systems	Executives, business, any department
C2 — Containers	Apps, APIs, databases, queues	DevOps, infra, data teams, engineers
C3 — Components	Modules inside one container	Developers
C4 — Code (optional)	Classes, ERD, sequence	Developers

Other properties from the slide: four zoom levels from big picture to code; **one level per audience**, executives to devs; **notation-free** — boxes, lines, labels; extra views — landscape, dynamic, deployment; tools — Structurizr, Mermaid, draw.io.

The instructor's verdict: C4 is the one he actually uses `[40:14]`

> "The framework that works with me and works beautifully — I don't even do all of it, I do the **first two layers**."

Why C3 and C4 can be left to AI `[40:14]`:

> "Nobody now goes and draws something from the code, the logical design, the components inside. Go to any repo — GitHub or your agentic coding tool — and say *explain the components here*. It will explain them, draw them, give you the class diagram, and even the pipelines. **Those two you can depend on AI for.**"

Why C1 and C2 are a must `[40:59]`:

- **C1 is a contract.** "You're describing the system at a very high level: this is inside the company, this is outside, here are the main components. This is **the document, the contract** between you and every department — the contract with business, with the network team, with the infra team, with security. You are documenting it."

- You may generate it with AI or by hand — but **you must review it, be sure it's 100% correct, and be able to edit it easily**.

- **C2 is where it gets physical.** Containers talking to each other over APIs; some processes are complex enough that you need a **data pipeline diagram or a sequence diagram** to explain them — authentication, or a multi-step generation flow.
  - "If you take a high-level diagram to the data team, they won't understand anything from it. They'll say 'what is this?'"
  - This is also the level where you talk to infra: "I want two nodes, a load balancer on top of them, and so on."

> **Rule of thumb from the session:** C1/C2 are stable and human-owned; C3/C4 are volatile and can be AI-generated and regenerated per use case.

Part 5 — The architecture development process

SLIDE 15

The activity diagram on slide 15, read left to right, with the instructor's narration at `[42:32]`.

5.1 The five steps

1. **Define Initial Scope and Context** — produces the *Scope and Context* input artefact. This is also where you can already give an initial answer or simple items.
2. **Engage Stakeholders** — not necessarily everybody from the first design. "The bigger the number of people in the scope, the more every one of them has an opinion and the thing never finishes." Sometimes the **key members** suffice: product manager, team, team lead — *the people whose decisions actually affect the outcome*. This is not about hiding anything; transparency still exists.
3. **Capture First-Cut Concerns** — produces *Stakeholder Concerns*. Dependencies on other teams, results, outputs, questions, assumptions, architecture principles.
4. **Define Architecture** — the central activity. Takes *Scope and Context* + *Stakeholder Concerns* as inputs; produces the **Architectural Description** and the **Guidelines and Constraints**.
5. **Create Skeleton System** — produces the **Skeleton System**.

5.2 The outputs — what you are actually accountable for

Output	What it is
Architectural Description	The design document you keep working on alongside the diagrams
Guidelines and Constraints	Your assumptions, your constraints, and what is *not* feasible
Skeleton System	The POC / MVP you build with your own hands

5.3 The loop `[43:18]`

> "You take all of this and go sit with each one, align them on it, then come back and digest the feedback. Feedback will come to you and you fold it back into your design. Do that once, twice — the solution is closed, everyone is satisfied, and then you sign off on implementation."

Part 6 — Views and perspectives

SLIDES 16-17

This is the technical heart of Session 1.

6.1 The six views — Slide 16

View	Question it answers
Functional	What are the functions/components, what are the flows?
Information	What is the data, where does it live, how does it move?
Concurrency	What happens when many things happen at once?
Development	How is the code organised for the people who build it?
Deployment	How is it physically deployed — nodes, networks, capacity?
Operational	How is it run in production — monitoring, alerts, support?

`[49:30]` The instructor's note on where attention naturally goes: "While designing, your focus is always **here** — on the functional view. Whatever level you're working on, context or container, you look at functionality, you look at the flow."

The others need deliberate effort: the information/data view (how the data will be done), concurrency (what if two users come in at once — how many hit the system per second), deployment (how we physically deploy this), and operational (when we go to production).

6.2 The perspectives / quality attributes — Slide 16

The slide lists: **Security, Performance, Availability, Usability** on one side and **Accessibility, Location, Regulation** on the other, plus *etc.* Slide 17 adds **Evolution**.

A perspective is **not** a view. It is a lens you drag across *all* the views.

6.3 The cross-section technique — the key idea of the session `[50:20]`

> The instructor's own metaphor, from Arabic school biology: a **cross-section** (*qitaa 'ardi*) — the diagram where you slice through a stem or a leg and look at it from the side.
>
> "You have laid out the system in layers: business, functional, deployment, data. Now I want to look at it **from the side**. Take security — start applying it across all of them. **You will be amazed** at what comes out."

The worked examples given in the session

View × Perspective	What it surfaces
Information × Security	Access control, access classes, object-level security. "Security on the data side, in its connection — that one will usually show up early anyway." *(Slide 17)*
Deployment × Security	The one you'd otherwise miss. `[51:07]` "When you focus on security in deployment you might notice — *wait, this container, or its DR, is outside Saudi Arabia*. Or the backup of this data might be outside Saudi. **So my system is not compliant.**"
Concurrency × Performance	Shared resources, blocking, queuing, coordination. `[52:40]` "If 4, 20, 50 people come in together…" *(Slide 17)*
Functional × Evolution	Extension points, flexible interfaces, meta-approaches. `[54:14]` "Product may give you a small use case that doesn't grow now — but you must look at whether it can give a use case in later phases that **fails to continue**." *(Slide 17)*
Functional × Security	Access issues, objective issues, privacy issues `[51:54]`

Evolution, expanded `[54:14]`

> "Don't design to the size of the function you have now. It's essential to know the **roadmap** that's coming, and the vision — and choose a design that goes with all of it." If the technology can't carry the next phase, **highlight it** to your manager now.

It is a mind-state, not a ritual `[51:07]`

> "You won't do this every time. But if your mind is present with it — if you put this **mind-set** with you — you won't need to look at the view to do it. When a new requirement comes to you, you'll start looking at it through these, and then through those in your head. You'll find yourself faster every time than the time before."

6.4 The budget consequence — why this is the architect's job `[52:40]`

A concrete example of Concurrency × Performance driving a decision that is not technical at all:

> **The SMS classifier.** "We built a machine-learning classifier that was supposed to classify **5,000 messages or SMS per second**. At that point we removed CPU from our heads entirely and started talking about GPUs. Then: *are you willing to pay?* — because the GPU will cost you around **$24,000 a year**." (That was 2022; earlier, 2017–18, at Etisalat.)

> "You, as an architect or an engineering lead, must also bring a **budget**. Someone will ask you for one. So you have to know what your system's ceiling and floor are."
>
> The wrong answer: *"I don't know, we'll try, God help us."* "You'll find yourself stuck in the wall, there's no budget, and the whole thing fails."

What the architect's output actually includes `[53:27]`:

- A calculation — "I use model X, its parameters are Y, therefore I need this GPU, this CPU, this DR, this backup." (You can do the calculation with AI.)
- **Capacity planning**.
- **Test cases** — you can propose test cases, making life easier for the people after you.

> "The output of your system is not a bit of code and that's it."

Part 7 — Architecture and Agile

SLIDE 18

7.1 From waterfall to sprints [55:01]

- In the waterfall era you did all the questions, took your time — a month — then came back; a design cycle for a use case could take **three to six months**.
- Since roughly 2010–2011 iterations shortened to two weeks, three weeks, a month — a sprint — and you're expected to do the **whole cycle** inside it.

7.2 Does agile change the architecture? [55:50]

Yes — it forces you to make the architecture **adaptive / marine**, and the answer comes from *which layer changes how often*:

Layer	Stability	Cadence
Context (C1) and Deployment (C2)	Will not change frequently or greatly	Done once, up front, high level — **project level**
Components, logical design, data pipelines	Dynamic	**Use case by use case** — epic level and story level

> "You put a few principles, a few fundamentals that no use case may violate — and you leave the lower layers some room."

7.3 The four levels on slide 18

Level	What architecture means here
Project level	The big one: context, deployment, principles. Done once.
Epic level	A use case's own design — components, data flows
Story level	Dynamic, left with room to move
Research / spike level	Investigation work that feeds back into the design

7.4 Principles as the stable layer [56:35]

> "We said, for example, that **all data must be inside Saudi / the UAE / Egypt**. That's it — that becomes a law for every epic. Anyone who comes to work on an epic must read it, or it must be a consideration."
>
> "And if you're using an AI design document, give it to your AI: *any principle or design or suggestion you give me — consider all the principles here.*"
>
> "That way you guarantee you've closed your thing, done it once, and the impact on it is small."

Part 8 — The architecture description

SLIDES 19-24

The architecture description is the written deliverable. Slides 20–24 give its headings.

> Naming is not worth arguing about `[57:26]`: "Some people call it one thing, some another — it doesn't matter. We know it's the design of the whole system."

8.1 Architecture intro and facts — Slide 20

The six standard headings:

1. **Problem statement / pain points**
2. **Goals / non-goals, solution assumptions**
3. **As-is architecture; explore the enterprise architecture**
4. **Architecture principles, key decisions and trade-offs**
5. **Assessment**
6. **Priorities**

8.2 High-level design — Slide 21

Applying C1 — context viewpoint

- System boundaries
- Source systems and destination systems
- Components breakdown

Applying C2 — deployment viewpoint

- Containers
- Nodes
- Ports and protocols
- Authentication / authorization
- Networking layer (inbound / outbound connections)
- Capacity planning (nodes, pods, DRs, CPUs, GPUs)

8.3 Information viewpoint / data pipelines — Slide 22

A checklist to run against every use case:

- C2 view
- Data pipelines — data flows
- Messaging and queues
- **Data velocity** — real-time, near-real-time, batch
- **Data size** — load / frequency
- **Data quality** — is any preprocessing required?
- **Data retention policy** / backups / replications
- **Data sources and ownership** — CRM, events, CDP, data lakes/warehouses, vector DB, SQL DBs
- **Data classification and privacy** — PII, PDPL, consent

> The instructor's own first question on any new use case [90:13]: *"Where does this data live, as-is?"* — "As long as I described the as-is system beforehand, it's easy. I won't rack my brain about where to get it from."

And immediately after that [90:13]: *"What is the nature of this transaction — does it need real-time, near-real-time, or batch?"* One stakeholder says it must answer within a second; another says five minutes to half an hour is fine; a third says "I want it every day at 12 at night."

8.4 Concurrency viewpoint — Slide 23

Driven by a **C2 sequence diagram / use case**. Questions to extract concurrency requirements and gaps:

- Processes, threads and workers mapped to containers
- **Sync vs async** interactions (API calls, events, queues)
- **Shared state and resources** (locks, transactions, caching)
- **Parallelism and scaling** (worker pools, partitions, autoscaling)
- **Rate limits** (LLM/API quotas, throttling)
- **Timeouts, retries and failure handling** (circuit breaker, DLQ — dead-letter queue)
- **Scheduling and orchestration** (cron, batch jobs, workflows)

8.5 Operational viewpoint — Slide 24

Observability layer

Tool	Role
Grafana	Resource utilisation
Loki	Logging aggregation
Prometheus	Alerts and OpenTelemetry metrics
Kibana	Dashboard & visualisation, analytics

MLOps layer

Tool	Role
Langfuse	AI logging and evaluations
MLflow	Model registry and lifecycle, experiment tracking, version control
Airflow, Flink, Prefect	Data preprocessing, schedulers and transformation

Model serving

Tool	Role
KServe	Kubernetes-native model serving, autoscaling, canary rollouts
BentoML	Package models as APIs, easy containerised deployment
vLLM	High-throughput LLM inference engine (PagedAttention, continuous batching)

Part 9 — The live design walkthrough

THE SECOND HALF OF THE SESSION

From [57:26] to [102:36] the instructor built the design document and the draw.io diagram live. This section reconstructs it. **This is the part you should redo yourself before the next session.**

9.1 Document skeleton

He opened a document titled *Marketing Automation — Design / Architecture* and laid down the headings first, so nothing gets lost while thinking:

```
Marketing Automation - AI Architecture

1. Problem statement
2. Goals / Non-goals
3. As-is architecture
4. Architecture principles & key decisions
5. To-be architecture
6. Use cases (one section each)
```

9.2 Problem statement — "What we are trying to resolve" [58:12]

The pain points, taken from the four functional use cases:

#	Pain point as written
1	The home page is **static offers** — needs to be **personalised** per customer or per segment
2	**Abandoned cart** — the cart is left open and we want to nudge the user
3	**Potential churn customers** — people likely to churn are not detected
4	**Cold start** — new users: nothing is known about them
5	**Agentic marketing** — generating new marketing campaigns by agentic AI (manual today)

9.3 Goals and non-goals [59:45]

Goal (as written live): design a personalisation / marketing system to **increase CSAT to ~70% after 6 months**.

And immediately — *how will it be measured?* **By measuring customer rating and feedback.** [60:30]

> The mechanism [61:17]: take the stars/rating people give, run **classification over the free-text feedback** to label it satisfied / dissatisfied, combine the two, and you get a simple satisfied-or-not signal. "This is real — we did this."

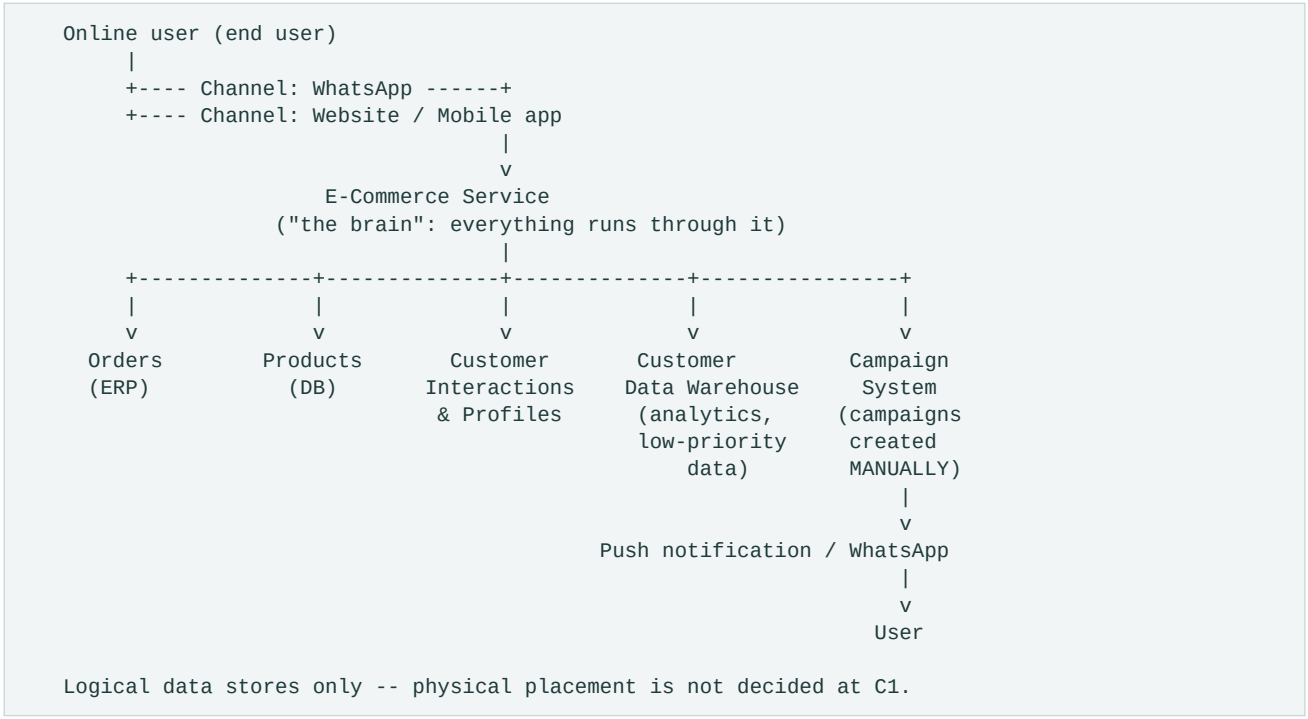
Non-goal (contributed by an attendee and accepted): *lower refunds.* [61:17] "We're saying this system's goal is **not** to reduce the volume of refunds. Excellent."

> Why non-goals matter as much as goals: they are the written agreement about what you will **not** be judged on at the end of the quarter.

Each pain point then becomes an **epic**, written in detail by the product manager in Jira/Confluence, and the architect does the assessment on top of it [62:04].

9.4 As-is architecture [62:04]

He drew the existing e-commerce system before adding anything. "Start from the actors / from the people."



Notes made while drawing:

- **Simplify first.** [65:09] "Let's make it mobile only for now, and next session we can add WhatsApp — because WhatsApp drags us into AI and today I want it pure and simple."
- The **e-commerce service is the brain** — everything enters through it and it stores the data.
- The data stores are drawn **logically**, not physically [65:55]: "one for interactions, profiles and so on, the other for products — whether both are in one database or two, here it's logical; I'm not talking physically."
- Home-page offers today: the service **retrieves fixed offers from Products** and displays them. "Personalisation — none yet. Next-best action — nothing there either."
- Shopping carts (completed and abandoned) live inside **Customer Interactions**.
- Churn signals also live there: `last order date`, `last login date`. [68:16] "Anything that tells you whether this person interacted with the system or exists in it."
- Campaigns: the user or the system goes to the campaign system, which **creates / lists campaigns manually**, and sends them out as push notification or over WhatsApp.

> Why the as-is drawing pays for itself [69:48]: "I take this view and put it in the design under *as-is*. **Why?** Because I'm making a reference for this system. Anyone who sits on this system afterwards understands what we have and what we don't have."
>
> And later, every time a use case comes up: "the data for the abandoned cart — you'll find it in transactions. Because I described the as-is system beforehand, it's easy; I won't rack my brain about where to get it from."

9.5 Architecture principles and key decisions [70:36]

Here he explicitly went back to **slide 17** and ran the **perspectives across the views** to generate principles.

From the Security / Compliance perspective

> Assumption stated out loud: this system runs in **Saudi Arabia** — "and this actually happened."

- **Data residency** — all data for the whole thing, including anything agentic we develop later, **stays inside Saudi Arabia**. Therefore **any model we use** must satisfy that too.
- **Data must be stored encrypted — in transit and on disk.**
- **GDPR / PDPL / PII** — respect data privacy.
- **Guardrails to comply** and to achieve clean text output.
- **Limit AI errors** — "any guardrail you wrote is to be applied across the whole system; the model should pass through it."
- Noted as "can be written later": **prompt injection**, **SQL injection** and other injection classes.
- **All internal systems sit behind a VPN** — "any internal service: the security team told you nobody may access it from outside."
- **All external systems are TLS-protected and use HTTPS**, with authentication (JWT or whatever mechanism you choose), plus **2FA** where appropriate.

From the Data / Information perspective

- **Retention policy** — e.g. *all customer data is kept for at least 5 years*.
- Where that number comes from `[80:04]`: from the **data team**, or from the **enterprise architecture**, or from **security & compliance**, or from the enterprise architects who own the whole company. "You'll find it written somewhere. You follow it."
- **Backups** — and note where the backup physically lands (this is the Deployment × Security trap).

From the Performance / Concurrency perspective `[73:41]`

These start as **questions and assumptions**, not answers:

- *How many users enter at peak hours?* → assumption: **50 concurrent users**.
- *Seasonality?* → **Ramadan, holidays** → peak can reach **100 concurrent users**.
- Decision that follows: either **autoscale** the infrastructure at those times, or **size for 100 from the beginning**. "If you don't support autoscaling, then from the beginning I'll size the project for 50 and for 100 — because if the holidays come later it will fall over."
- **Rate limiting** (contributed by an attendee and accepted).

The capacity decision that follows `[76:09]`

Dropping into the **deployment view**, the sizing becomes concrete:

```
2 nodes, active-active
4 cores CPU each
16 GB RAM each
2 pods per node

If on Kubernetes:   min 4 GB  /  max 16 GB   -> let HPA scale up and down
```

> "You went and talked to the infra team, looked at the data volume and the transaction volume, and took a decision based on the concurrency that came to you."

> **These principles are not decoration.** `[73:41]` "We haven't started, we haven't written anything, we haven't designed — and these things came out. Everything that comes out early changes which model you go with and which design you choose."

9.6 To-be architecture — the layers that get added [81:39]

Observability layer

Added as a new layer over the system, containing:

- **AI-as-judge / LLM-as-judge** for accuracy measurement.
- **A small model for security and guardrails** that scans transactions.

Where the security scanner sits — a real trade-off [82:26]

Two placements, and he deliberately showed both:

Placement	How it works	Trade-off
Inline, at the API layer	Every transaction passes through the scanner on its way through	Catches violations **before** they reach the customer, but **adds latency** — "if you do this with every API, you'll notice things slow down"
Offline, at the data layer	Scans the stored datasets after the fact ("scan for PII/breaches")	**No latency cost**, but "some things will get through — they reached the customer and you haven't caught them yet"

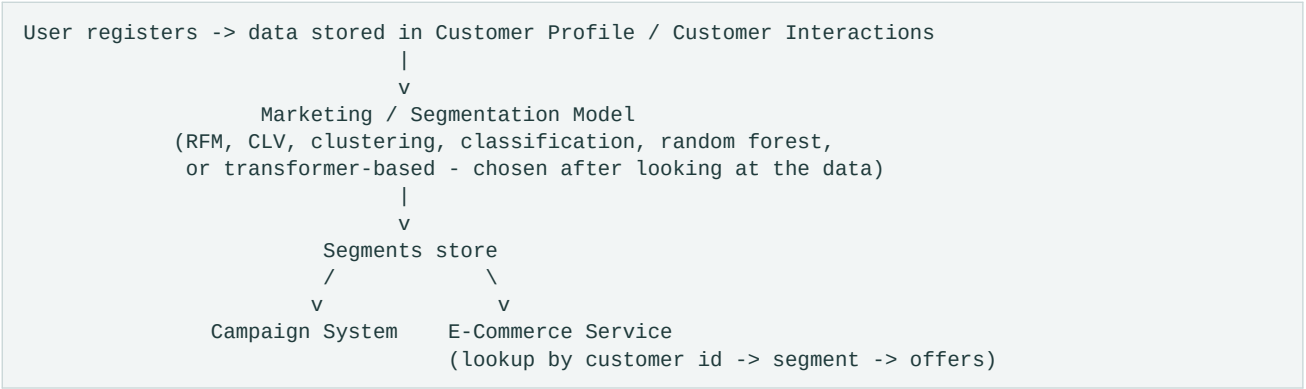
> "So there are several layers to this. Some things are rule-based and don't need AI at all. You add according to the need."

Guardrails as a cross-cutting rule [84:02]

"Any guardrail you put is **to be applied to the whole system**. The model should pass through it, somehow — you reconcile it against this. And we'll bring it back in the agentic AI session."

9.7 Use case 1 — Segmentation (the base for everything else) [84:02]

Decision: no deep AI today. Start with segmentation.



Points made while drawing:

- **Model choice is deferred, deliberately** [84:47]: "Here we're not picking the model. In your reality you *will* choose the model — but not at this level."
- Candidate families named: **RFM**, **CLV (customer lifetime value)**, segmentation models generally — "there are a million small models in marketing."
- **Cost awareness** [85:34]: "An LLM has a cost, of course. Some things need GPUs, some run on CPUs."
- **How you actually choose** [87:51]: "In our team we do a small **research spike**, look at the data, its nature and its format. You need to look at the data first."
- This can run **offline / after the fact** — "no problem at all" [86:19].

- Output: a **segments** table, possibly in the same database or a separate one, consumed by the campaign system and the e-commerce service. At request time: "you know the customer logged in, you pull his data by ID, get the segment he belongs to, and start displaying accordingly."

The naming discipline — a C1 rule worth memorising [87:51]

> "This view is **context** — you are not technical here. So the **names you write must be semi-technical**. Make sure whoever reads it understands. If I write *clustering*, he won't understand what clustering is. You have to write it in **his** terminology — and implicitly, in other documents, you'll do k-means or whatever clustering technique you use."
>
> Hence: the box says **"Segmentation"**, not **"K-Means"**.

9.8 Use case 2 — Personalisation / recommendation engine [88:39]

Drawn as a **black box labelled for next session**.

The distinction he drew [89:26]:

- **Segmentation gives you a recommendation for the whole group** — "the group of people who resemble each other, who buy leather bags, for example."
- **A recommendation engine gives a personalised product to a specific user** — "like when you open Amazon or Noon and find things coming to *you*."

Once the recommendation engine exists, the campaign system and the other use cases can take their input from it instead of from segmentation — "it can take from segmentation, no problem, but if you want per-user, you need the engine."

9.9 Use case 3 — Abandoned cart / next-best action [90:13]

Where the data is: already answered by the as-is — in **Customer Interactions / Transactions**.

Velocity decision: this one can be batch.

> The Etisalat precedent [91:01]: "We used to update the mobile offers **every day** — not in real time, not when the user opens the app. The requirement was: at 12 at night the model updates, re-ranks, and whoever opens the app finds new offers that last 24 hours. **Later** they said they wanted it real-time, so we built the recommendation engine with a personalisation model plus ranking, choosing products per user."

The design, as drawn:

```
Nightly ETL job (00:00)
   -> scan Customer Interactions for carts abandoned in the last 24 hours
   -> generate a personalised abandoned-cart notification
   -> send to WhatsApp channel  (or push notification service -> mobile/web)
```

Written on the box: `scan last 24 hours AND send message to WhatsApp`.

Implementation note [91:48]: today this can be **rule-based** — no AI at all. Later it belongs in the recommendation engine, "better that it selects, because it has all the information and chooses the next-best action based on it — it's very hard to do as a rule."

Placement note [93:19]: he chose to attach it to the recommendation engine side even though it isn't AI yet, "because later on it can work with AI" — and because the recommendation engine is a big thing that can start with small pieces and become personalised later.

9.10 Use case 4 — Churn prediction [94:52]

Model actually used in production: *random forest*. "Random forest — because the nature of our data was a bit like that, so it starts classifying people like a tree, brings out the people who resemble each other."

The churn window — and the most transferable idea in this section:

> **The model must be domain-specific.** [95:39]
> "Churn in a company like Etisalat is not the same as churn in a car company. Etisalat is an app you open every day — so if someone sits three days or a week without opening it, something is wrong. But a car company? It's completely natural that they open it once a month, or once every three months."
> "So the window depends on the **industry** and the **domain** you're working in. In some companies the window is three days; in others a week."

Scheduling rule [98:00]:

```
ETL job runs every N hours (e.g. every 24h)
   -> scans the data
   -> runs the churn model
Rule: the model must run at an interval SHORTER than the minimum churn window.
      small window  -> run daily
      large window  -> run weekly
```

Output feeds fixed rules → generate next-best action → select / rank offers.

> On method: "you sit and do trial and error until you get there, because the thing isn't fixed."

9.11 Use case 5 — Cold start [99:31]

"Simply: you have people who are still new. **This one needs no AI at all.**" Three options, in increasing sophistication:

1. **Fixed / preferred offers** — "five offers, or whatever; anyone who comes in finds them."
2. **On-the-fly segmentation** — "if your latency allows, run the segmentation model on him alone, put him in a segment, rank, and send him the offers."
3. **Demographic data** — use what you *do* know about someone who has no interactions yet.

Implicit vs explicit interactions — a definition worth keeping [100:18]

Type	What it is	Examples
Explicit	What the user told you directly	Demographics given at registration, stated interests
Implicit	What you collect from behaviour	Time spent on each screen, which screens were opened, last purchase, purchase history, installs, app opens

For a cold-start user you only have the **explicit** side — so you rank or segment on that, and the implicit signal accumulates afterwards.

> Documentation note [101:50]: "Every one of these use cases gets its own section in the document. So you separate them: you'll use the segmentation model first, then take its data and present it here to the models."

9.12 Use case 6 — Agentic marketing

Deferred. [101:50] "That's a big story — we'll talk about it next time, or the time after."

9.13 What the course output is [102:36]

> "Architecture isn't theoretical — you *do* implement the architecture. But the output of this course, as we agreed, is that we **build the whole architecture together**." Writing the code for each model is optional, and a good fit for the mini project if you want to take it further yourself.

Part 10 — Architecture governance

SLIDES 25-26

10.1 The five key points — Slide 26

1. **Always connect and relate to business goals.**
2. **Principles and standards** — approved technologies, patterns, reference architectures.
3. **Architecture Review Board (ARB)** — reviews designs at key gates, approves exceptions.
4. **Decision records (ADRs)** — context, options, trade-offs, owner.
5. **Light-touch automation** — guardrails over gates, automated checks in CI/CD (optional).

10.2 Where the documents actually live `[107:59]`

An attendee asked whether the file structure depends on the company. The answer:

- In Confluence: **every team has a space, every project has a space.** You link your epics and items to it.
- On a startup/GitHub project: you link from the project.
- **It does not matter where you write it.** `[108:44]` "In the end you have a set of headings — the dotted-line points. You'll mention them on an epic, a story, in Confluence, in a Word doc. What matters is that **all those headers exist**, or at least that you verify they exist."
- Keep links bidirectional: the host added that they **mention the document in the Jira ticket comments**, so that opening the ticket gets you back to the design.
- **A release document** matters too, and if it is essential it can live **in the repo itself**.

Part 11 — Architecture & agentic coding

SLIDES 27-28

Slide 28 lists three items: *Claude as an architect*, *skills needed*, *swarm of agents*. The session's treatment `[13:09]`:

11.1 Do it manually first

> "I prefer to do something **manual first, by hand**."

11.2 Never fully autonomous — always plan mode

> "In agentic mode there's a fully autonomous mode where you give it something and say *live your life, do a design*. **I don't do that.** I always use **plan mode**."
>
> "Plan mode's benefit is that **I tell it my principles, I tell it my rules** — as if I'm letting it work as an architect, but **I'm the one leading**. So it doesn't give me anything with surprises in it."

The instructor's phrase for his own practice: *"I let the AI work with me while I do the system design"* — not instead of him.

11.3 Skills to give the agent

Two concrete examples named:

- A **release skill**.
- An **architecture principles skill** — "so that while it designs anything I give it, it must apply the rules I told it. It shouldn't be left alone with itself."

And from the agile section `[56:35]`: *"If you're using an AI design document, give it to the AI: any principle, design or suggestion you give me — consider all the principles here."*

11.4 The division of labour with C4

Level	Who does it
C1 — Context	Human-owned. You may generate it, but you **must** review it, confirm it is 100% correct, and be able to edit it easily. It is a contract.
C2 — Containers / deployment	Human-owned, same reasoning
C3 — Components	Can be generated from the repo by an agentic tool
C4 — Code / class diagrams / pipelines	Can be generated from the repo by an agentic tool

11.5 Where agentic-coding cost belongs `[103:22]`

A question on budgeting agentic coding. The answer separates two budgets:

Budget	Owner	Example
Engineering budget	Company-wide	Cursor-style licences bought for the whole company
Product architecture budget	Your design document	"I expect to use ~3 million tokens over the next three months because of this application", or "I need GPUs for a year, costing $12,000"

> "You do **not** put the agentic coding licences in your product architecture — but it is highly preferable that you know and surface the product's own AI cost."

Part 12 — Q&A from the session

Q: Can you recommend a book? [105:40]

- **"Software Systems Architecture"** — by Nick Rozanski and Eoin Woods. This is the book the slides on **views and perspectives** come from (slides 16 and 17 reproduce its Figures 4-1 and 4-2). "I read it years ago but it's still valid."
- **Substack** — "like Medium. Very good, and there are people on it who put up a system design and explain the use cases. It's more **practical**; the book is a bit theoretical."

Q: Is AI architecture different from software architecture? [107:12]

> "**No, not at all.** It's only that when we come to design an AI system there are **extra components** added."

Where they appear:

- **Presentation layer** — a new kind of interface: chatbots, WhatsApp, channels, on top of the old front end / back end / mobile app.
- **Back end** — LLMs and "brains".
- **Everywhere** — observability and evaluation.

"That's what we'll talk about gradually, lecture by lecture: how to apply the AI engines onto the system."

Q: CTOs push for speed and short timelines — does this framework work in startups? [104:07]

> "It is *made* for that."

- An MVP does not mean building everything. "I can ship a small MVP, quickly — it won't take long."
- But while shipping it, **extract the principles and the risks** so it doesn't blow up later.
- **Push back and reduce the scope** of the first release as much as you can. A smaller scope leaves you room to put the principles in.
- You do it **once**; the next ones are faster because "you did it the first time and you probably won't change it much afterwards."
- If they insist anyway: **your duty is to highlight the risk** — *"there's a risk: if you ship X, we may violate PDPL."* Say it, and let them take the decision. "In the end you give trade-offs."

Q: I'm a junior — how do I digest this before the next session? [109:30]

The instructor called it "a very good question". Three answers:

1. **Do the assignments.** The shared document is intentionally headings-only — "I'll never be able to fill the whole document during the course."
2. **Take one section at a time and apply it to a project you already work on.** "Take a system **you know** — don't take something strange to you — and start: if I were to build it with these concepts, how would I do it? Would it stay the same, or would I change something?"
3. **Read the source code of big projects** (the instructor's own early habit, pre-AI), and read the books, and apply them in your day-to-day.

> A caution for juniors [110:16]: **"Don't try to impose what you're doing on the rest of the team."** Bring it gradually, as a suggestion.

And the framing of why this matters more than code [111:02]: "**The problem was never in the code.** The problem is that people don't apply these rules — they don't extract the non-functional requirements, they ship

anything into production and then get surprised by all these problems."

Q: How do I transition from senior to tech lead / solution architect? [111:50]

Explicitly deferred as out of scope for the session — the instructor offered to answer it offline by voice message, and the host collected it for a questions document to be reviewed at the start of the next session.

Part 13 — W1 mini project (assignment)

SLIDE 36

13.1 The brief

Mini project: a governance pack for the Marketing Automation with AI & Agents system.

1. Write **5 architecture principles**, each **linked to a business goal**.
2. Write **3 ADRs** (Architecture Decision Records) — e.g. API vs self-hosted LLM, vector DB choice, real-time vs batch — with **options, trade-offs, decision**.
3. Build an **ARB checklist**: security, privacy (PDPL), evals, cost, human oversight.
4. *(Optional)* Add one **automated guardrail** — a CI check, or an agent reviewing a design against your principles.
5. **Deliver:** one wiki page and one draw.io diagram, **max 2 pages**.

13.2 How to attack it, using this session's material

For the 5 principles

Take them straight from the live walkthrough (§9.5) and make sure each one names its business goal:

Principle	Linked business goal
All data stored and processed inside KSA; models must satisfy residency	Legal operation / avoid PDPL fines
Data encrypted in transit and on disk; PII handled per PDPL	Customer trust → CSAT
All internal services behind VPN; external endpoints TLS + JWT (+2FA)	Protect revenue-generating channels
Guardrails applied system-wide to every AI output	Explainability and business control
Design to the roadmap, not just today's use case (evolution)	Sustained take-rate growth across phases

For the 3 ADRs

Use the trade-offs that actually appeared in the session — they are already half-written:

1. **Real-time vs batch for next-best action.** Options: nightly ETL at 00:00 (the Etisalat precedent) vs real-time recommendation engine. Trade-off: cost and complexity vs freshness.
2. **Inline vs offline security/PII scanning.** Options: scan at the API layer (catches everything, adds latency) vs scan the datasets after the fact (no latency, some leakage). See §9.6.
3. **Fixed sizing vs autoscaling for seasonal peaks.** Options: size for 100 concurrent users permanently vs autoscale from 50 to 100 during Ramadan/holidays. Trade-off: steady cost vs elasticity and cold-start risk.

For the ARB checklist

Build it from the perspectives grid (§6.3) plus the capacity/budget duty (§6.4):

- **Security:** residency, encryption at rest/in transit, VPN/TLS/JWT, injection classes, DR location.
- **Privacy (PDPL):** PII classification, consent, retention policy, backup location.
- **Evals:** how model metrics map to the business KPI; LLM-as-judge in the observability layer; feedback loop defined.
- **Cost:** capacity planning, GPU/CPU sizing, token forecast, annual budget figure.
- **Human oversight:** which stages must stay human (remember the handwriting/OCR case), explainability answer ready for business.

Part 14 — Glossary of the session's vocabulary

Term	Meaning as used in this session
AD	Architecture Description — the written design document
ADR	Architecture Decision Record: context, options, trade-offs, decision, owner
ARB	Architecture Review Board — reviews designs at gates, approves exceptions
As-is / to-be	The architecture as it exists today vs the target architecture
C1 / C2 / C3 / C4	The four C4 levels: context, containers, components, code
Cold start	A user (or item) about which the system has no interaction history
CLV	Customer Lifetime Value — a segmentation/marketing model family
CSAT	Customer satisfaction — the business-facing KPI used in this project
DLQ	Dead-letter queue — where messages go after repeated failures
DR	Disaster recovery (site) — and a residency trap if it sits in another country
Explicit / implicit interactions	Data the user gave you vs data collected from their behaviour
Feature flag	A switch that ships code dark, enabling limited rollout
GA	General availability — full rollout after a limited-audience test
Guardrails	Checks applied to AI inputs/outputs to keep them compliant and clean
HPA	Horizontal Pod Autoscaler (Kubernetes min/max replica scaling)
MVP	Minimum viable product — but "360 degrees": functional *and* non-functional
NBA	Next-best action
Non-goal	Something explicitly declared out of scope for this system
PDPL	Personal Data Protection Law (Saudi Arabia)
Perspective	A quality attribute applied *across* views (security, performance, …)
PII	Personally identifiable information
POC / skeleton	The small thing you build with your own hands to validate the design
RFM	Recency, Frequency, Monetary — classic marketing segmentation model
Spike	A timeboxed research task when the answer isn't known yet
Take-rate	Share of offers/recommendations actually acted on
View	One structural angle on the system (functional, information, …)
Viewpoint	The template/audience definition behind a view

Part 15 — Self-check before the quiz

Answer these from memory; each maps to a section above.

1. Name the six views and the eight perspectives. Which **pair** surfaced the data-residency violation in the live walkthrough? *(§6.1, §6.3)*
2. Why is C1 described as a **contract**, and what makes C3/C4 safe to generate with AI? *(§4.3, §11.4)*
3. Business says "accuracy 90%". What is wrong with reporting that, and what do you report instead? *(§2.2)*
4. Who owns **Why/What** and who owns **How/When**? Give one failure mode from mixing them. *(§3.4)*
5. You've been handed a requirement you believe is infeasible. What are the two paths, and what must you do in **both** cases? *(§3.5)*
6. The segmentation box on the C1 diagram — why is it labelled "Segmentation" and not "K-Means"? *(§9.7)*
7. Your churn window is 3 days. How often must the ETL job run, and why does the window differ between a telecom and a car dealer? *(§9.10)*
8. Give the two placements for a PII/security scanner and the trade-off between them. *(§9.6)*
9. What three artefacts does the architecture development process produce? *(§5.2)*
10. Peak concurrency rises from 50 to 100 during Ramadan. Name the two design responses and when you'd pick each. *(§9.5)*
11. Which architecture layers are stable across sprints, and which are redone per use case? *(§7.2)*
12. List three things an architect must deliver besides diagrams. *(§6.4)*

Appendix — Session 1 at a glance

Section	Slides	Timestamps
Intro, course framing, logistics	1–5	00:00 – 14:00
The architect's personality & culture	6	14:43 – 20:55
The project and business requirements	7–9	20:55 – 34:50
Frameworks: TOGAF, Zachman, C4	11–14	34:50 – 42:32
Architecture development process	15	42:32 – 48:43
Views and perspectives	16–17	48:43 – 55:01
Architecture & agile	18	55:01 – 57:26
Live design walkthrough	19–24	57:26 – 102:36
Q&A	—	102:36 – 113:22
Governance, agentic coding, next sessions	25–36	referenced throughout

Next session: AI system building blocks — data pipelines and feature stores, the model layer, training vs inference infrastructure, serving and orchestration, MLOps/LLMOps and observability. The personalisation / recommendation engine black box from §9.8 gets opened there.